

```

import matplotlib
from matplotlib import pyplot as plt
import numpy as np
import rebound
import mpl_toolkits.mplot3d.axes3d as p3
import matplotlib.animation as animation
from mpl_toolkits import mplot3d
from tqdm import tqdm
from astropy import constants as c
from astropy import units as u
from scipy.signal import find_peaks

```

Problem 1: restricted 3-body problem [6 points]

Place a planet at $a_p=1$ au with $e_p=i_p=0$ and $m_p=10^{-3}M_\odot$. We shall place a test particle near L4: 60° ahead of the planet, $a=1$ au and $i=e=0$.

a) Show the evolution for 50 orbits (years) in the co-rotating frame $\{\mathbf{x}'\}$. It's easiest to perform the rotation (We can also use the mean motion $n=2\pi/P$ to define the rotation matrix. Such rotation should consider that the total mass is $1M_\odot+m_p$) as:

$$\begin{aligned} \{\mathbf{x}'\}(t) &= \begin{bmatrix} x_p(t) & y_p(t) \\ -y_p(t) & x_p(t) \end{bmatrix} \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} \end{aligned}$$

where the determinant of the matrix is $x_p^2 + y_p^2 = a_p^2 = 1$ (circular orbit) as required by a rotation matrix. You should get a tadpole orbit in this reference frame.

Primero podemos definir una función para la trayectoria que debe seguir la masa de prueba. Ésta viene dada por:

```

def tp_trajectory(a=1,tt=1500,mass = 1e-3):
    sim = rebound.Simulation()
    sim.G = c.G.value
    sim.units = {'AU', 'Msun', 'yr'}
    sim.add(m=1)
    sim.add(m=mass,a=1,e=0,inc=0)
    sim.add(m=1e-30,a=a,f=np.deg2rad(60),inc=np.deg2rad(0))
    sim.move_to_com()
    xp = []
    yp = []
    zp = []
    xt = []
    yt = []
    time = []
    for i in np.linspace(0,sim.particles[1].P*tt,300000):
        sim.integrate(i)

```

```

#fig, ax = rebound.OrbitPlot(sim,color=True)
time.append(i)
xp.append(sim.particles[1].x)
yp.append(sim.particles[1].y)
zp.append(sim.particles[1].z)
xt.append(sim.particles[2].x)
yt.append(sim.particles[2].y)
#display(fig)
#plt.close(fig)
#clear_output(wait=True)
time,xp,yp,xt,yt = [np.array(lst) for lst in [time,xp,yp,xt,yt]]
return (xp*xt+yp*yt,-yp*xt+xp*yt)

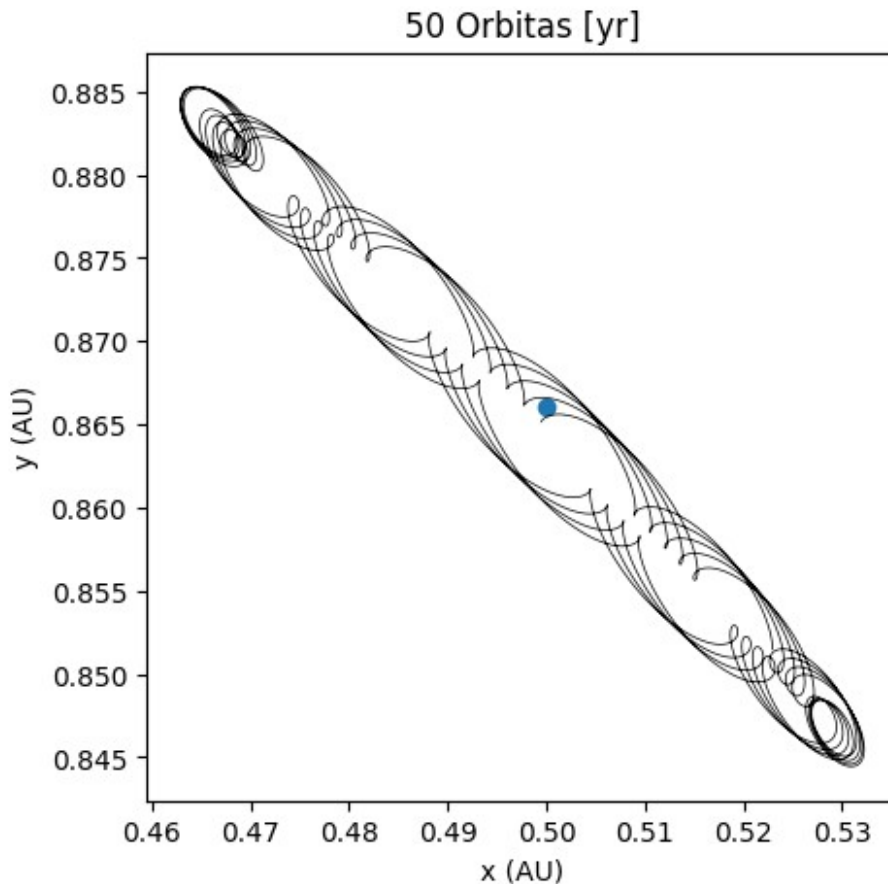
```

Ahora se muestra la evolución para 50 orbitas

```

fig, (ax1) = plt.subplots(1,1,figsize=(5,5),dpi=100)
ax1.plot(tp_trajectory(1,tt=50)[0],tp_trajectory(1,tt=50)
[1],linewidth=0.5,c='black')
ax1.scatter(np.cos(np.pi/3),np.sin(np.pi/3))
ax1.set_xlabel('x (AU)')
ax1.set_ylabel('y (AU)')
ax1.set_title('50 Orbitas [yr]');

```



Al realizar las orbitas en torno al punto de Lagrange 4 (L4) se observa una orbita de tipo Tadpole mencionada en el enunciado.

b) Gradually increase the test particle semi-major axis from $a=1$ au to $a=1.04$ au and show the evolution for 1000 orbits (all in one figure for better comparison). Describe.

```
a0 = tp_trajectory(1,tt=1000)
a1 = tp_trajectory(1.01,tt=1000)
a2 = tp_trajectory(1.02,tt=1000)
a3 = tp_trajectory(1.03,tt=1000)
a4 = tp_trajectory(1.04,tt=1000)
a5 = tp_trajectory(1.05,tt=1000)

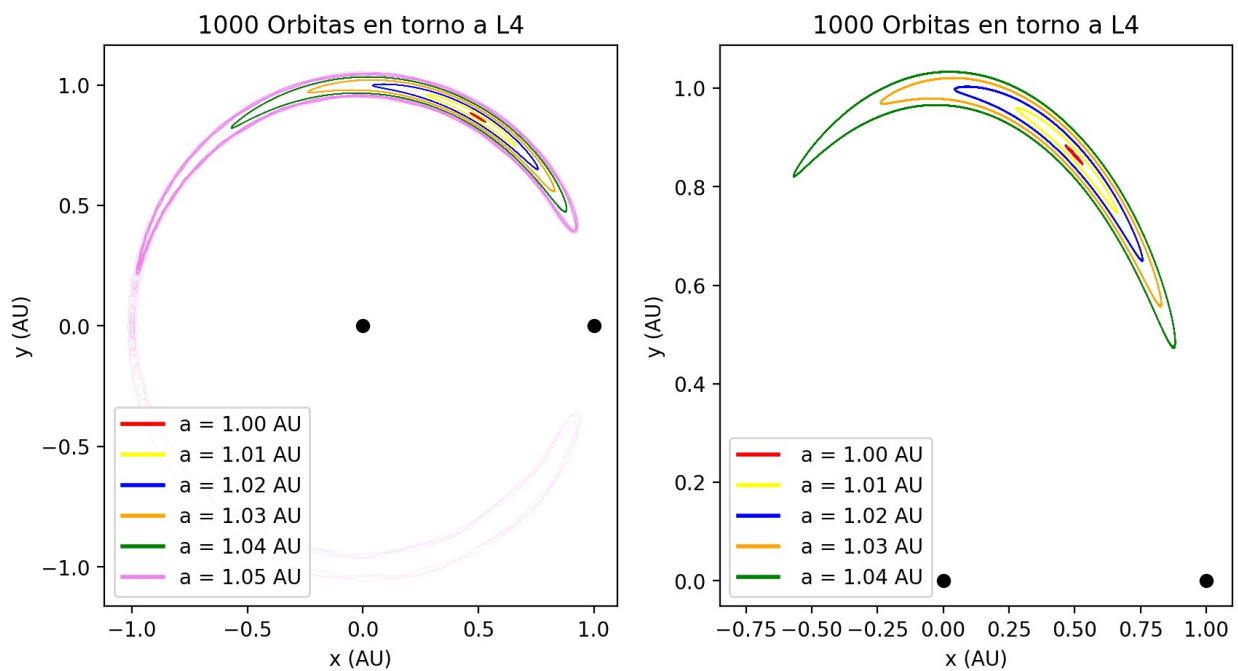
fig, (ax1,ax2) = plt.subplots(1,2,figsize=(10,5),dpi=200)
ax1.plot(*a0,linewidth=0.05,label='a = 1.00 AU',c='red')
ax1.plot(*a1,linewidth=0.05,label='a = 1.01 AU',c='yellow')
ax1.plot(*a2,linewidth=0.05,label='a = 1.02 AU',c='blue')
ax1.plot(*a3,linewidth=0.05,label='a = 1.03 AU',c='orange')
ax1.plot(*a4,linewidth=0.05,label='a = 1.04 AU',c='green')
ax1.plot(*a5,linewidth=0.05,label='a = 1.05 AU',c='violet')
ax1.scatter(1,0,c='black')
ax1.scatter(0,0,c='black')
ax2.plot(*a0,linewidth=0.05,label=' a = 1.00 AU',c='red')
```

```

ax2.plot(*a1,linewidth=0.05,label=' a = 1.01 AU',c='yellow')
ax2.plot(*a2,linewidth=0.05,label=' a = 1.02 AU',c='blue')
ax2.plot(*a3,linewidth=0.05,label=' a = 1.03 AU',c='orange')
ax2.plot(*a4,linewidth=0.05,label=' a = 1.04 AU',c='green')
ax2.scatter(1,0,c='black')
ax2.scatter(0,0,c='black')
ax2.set_xlim(-0.85,1.1)

for ax in [ax1,ax2]:
    ax.set_xlabel('x (AU)')
    ax.set_ylabel('y (AU)')
    ax.set_title('1000 Orbitas en torno a L4')
    leg = ax.legend(loc='lower left')
    for legobj in leg.legendHandles:
        legobj.set_linewidth(2.0)

```



Podemos ver que la trayectoria de la partícula se va alargando a medida que aumentamos la masa. Para la masa con $a = 1.05$ AU, vemos que esta se escapa hasta el L3 y luego alcanza a llegar a L5

c) Repeat b , not now gradually increasing the mass of the planet to $0.03 M_{\odot}$ (fix $a=1$ au). Describe.

```

m0 = tp_trajectory(mass = 0.001,tt=1000)
m1 = tp_trajectory(mass = 0.003,tt=1000)
m2 = tp_trajectory(mass = 0.007,tt=1000)
m3 = tp_trajectory(mass = 0.01,tt=1000)

```

```

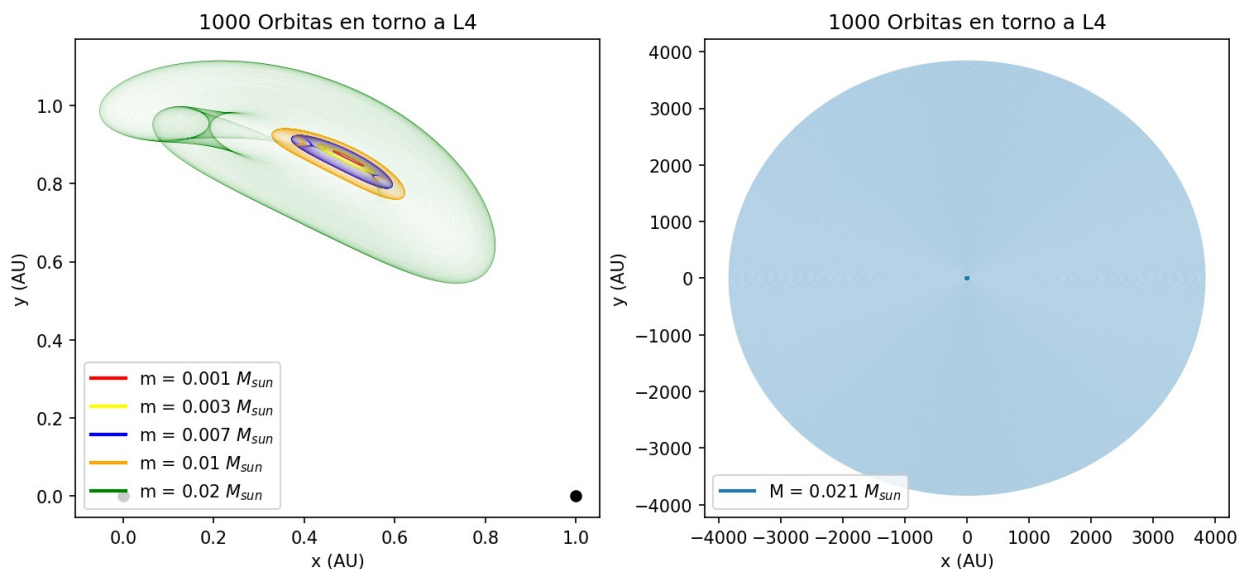
m4 = tp_trajectory(mass = 0.02, tt=1000)
m5 = tp_trajectory(mass = 0.021, tt=1000)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5), dpi=150)
ax1.plot(*m0, linewidth=0.01, label = r'm = 0.001 $M_{\text{sun}}$', c='red')
ax1.plot(*m1, linewidth=0.01, label = r'm = 0.003 $M_{\text{sun}}$', c='yellow')
ax1.plot(*m2, linewidth=0.01, label = r'm = 0.007 $M_{\text{sun}}$', c='blue')
ax1.plot(*m3, linewidth=0.01, label = r'm = 0.01 $M_{\text{sun}}$', c='orange')

ax1.plot(*m4, linewidth=0.01, label = r'm = 0.02 $M_{\text{sun}}$', c='green')
ax1.scatter(1, 0, c='black')
ax1.scatter(0, 0, c='black')
ax2.plot(*m5, linewidth=0.05, label = r'M = 0.021 $M_{\text{sun}}$')
for ax in [ax1, ax2]:
    ax.set_xlabel('x (AU)')
    ax.set_ylabel('y (AU)')
    ax.set_title('1000 Orbitas en torno a L4')

leg = ax.legend(loc='lower left')
for legobj in leg.legendHandles:
    legobj.set_linewidth(2.0)

```



Podemos ver que al aumentar la masa, las orbitas tienden a ser mas anchas y no elongadas como en el caso anterior. Para una masa aproximadamente mayor a 0.02, se pierde la estabilidad del sistema resultando en la expulsión de la partícula del sistema.

d) (Bonus +2): try to set-up a horse-shoe orbit (+2 pt.)

Para esta parte podemos simplemente añadir los parámetros que conoemos para las orbitas de tipo horse-shoe (Ω, ω, inc)

```

def p22(a=1,tt=1500,mass = 1e-3, test_mass=1e-30, test_ecc=0,
test_inc=0,test_Omega=0, test_omega=0, test_M=0):
    sim = rebound.Simulation()
    sim.G = c.G.value
    sim.units = {'AU', 'Msun', 'yr'}
    sim.integrator = 'IAS15'
    sim.add(m=1)
    sim.add(m=mass,a=1,e=0,inc=0)
    sim.add(m=test_mass,a=a, inc=test_inc, omega=test_omega,
Omega=test_Omega,e=test_ecc, f=np.deg2rad(60))
    sim.move_to_com()
    xp = []
    yp = []
    zp = []
    xt = []
    yt = []

    time = []
    for i in np.linspace(0,sim.particles[1].P*tt,1000000):
        sim.integrate(i)
        time.append(i)
        xp.append(sim.particles[1].x)
        yp.append(sim.particles[1].y)
        zp.append(sim.particles[1].z)
        xt.append(sim.particles[2].x)
        yt.append(sim.particles[2].y)

    time,xp,yp,xt,yt = [np.array(lst) for lst in [time,xp,yp,xt,yt]]
    return (xp*xt+yp*yt,-yp*xt+xp*yt)

fig, (ax2) = plt.subplots(1,1,figsize=(5,5),dpi=150)

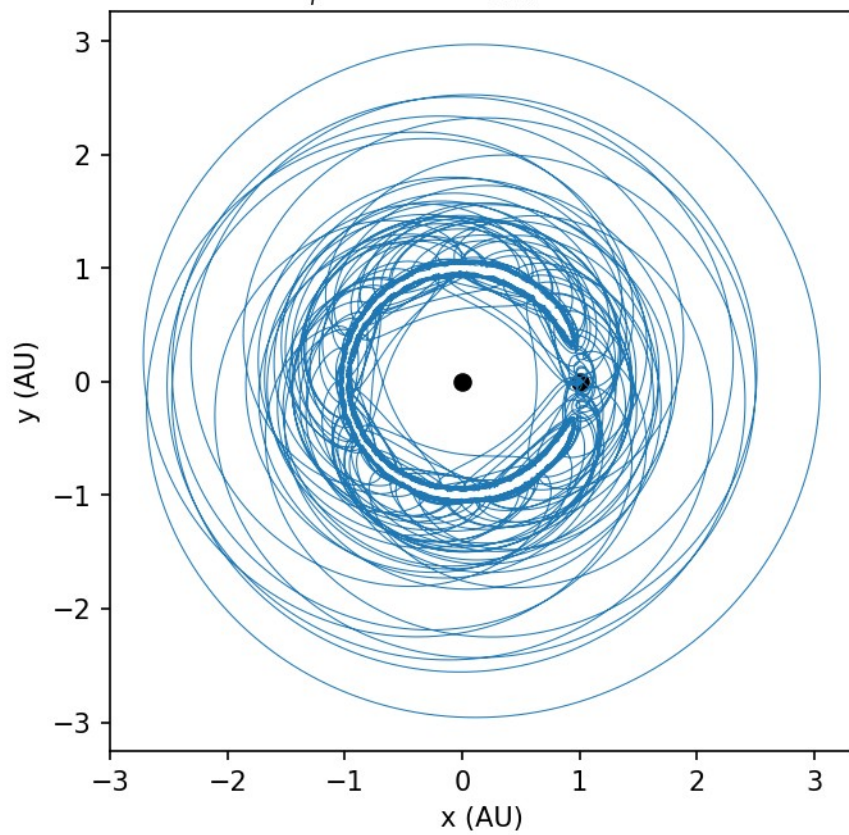
hs = p22(mass = 0.001, a=1.055, tt=500)
ax2.plot(*hs,linewidth=0.5)
ax2.scatter(0,0,c='black')
ax2.scatter(1,0,c='black')
for ax in [ax2]:
    ax.set_xlabel('x (AU)')
    ax.set_ylabel('y (AU)')
#ax1.set_title(r'$2002 AA_{29}$')
ax2.set_title(r'$M_p$ = 0.001$M_{sun}$, $a=1AU$')
fig.suptitle('Horse-show Orbit (500 orbitas)')

Text(0.5, 0.98, 'Horse-show Orbit (500 orbitas)')

```

Horse-show Orbit (500 orbitas)

$$M_p = 0.001M_{sun}, a = 1AU$$



A pesar de que muestra ciertas inestabilidades en la orbita, podemos ver de forma general la orbita tipo Horse-shoe que se pide. Los parámetros fueron hayados de forma arbitraria

Problem 2: Jupiter shepherding hazardous asteroids [9 points]

In class 2, we briefly discussed a planet formation model called the Grand Tack hypothesis, in which Jupiter migrates inwards and then outwards. This migration depletes the disk of solids down to the location of Mars, thus (possibly) explaining its small size.

In this exercise, we emulate the shepherding of asteroids by an inwardly migrating Jupiter and test some of the theoretical results of resonance capture. To set up the problem, use Reboundx and set a Jupiter-mass planet with eccentricity $e_J=0.01$ and semi-major axis

$$a_J(t) = 5 \text{ au} \exp(-t/\tau_a)$$

with $\tau_a=10^5$ yr. Place a mass less asteroid (or very low mass body) initially at $a_{\text{ast}}=2.8$ au with an excentricity of $e_{\text{ast}}=0.01$, and evolve the system for 10^4 yrs.

(a) [3 points] Plot the evolution of the period ratio $P_{\text{J}}/P_{\text{sat}}$. Is the system trapped into a mean-motion resonance? If so, show the evolution of the critical angle φ_{s} (The slow angle can be written as $\varphi_{\text{s}}=(m-1)\lambda_{\text{sat}}-m\lambda_{\text{jup}}+\varphi_{\text{sat}}$) and show that it librates (you may use $\cos \varphi_{\text{s}}$ for better visualization).

A continuación hay que ocupar reboundx para poder cambiar el valor de tau_a. Y la evolución se extendió de 10^4 para entender de buena forma lo que sucede.

```
import reboundx

sim = rebound.Simulation()
sim.units = ('yr', 'AU', 'Msun')
sim.integrator = "whfast"
sim.dt = 0.001
sim.add(m=1.)
sim.add(a=5, m=1e-3, e=0.01)
sim.add(a=2.8, m=1e-9)
sim.move_to_com()
ps = sim.particles
rebx = reboundx.Extras(sim)
mod = rebx.load_operator("modify_orbits_direct")
rebx.add_operator(mod)
ps[1].params["tau_a"] = -10**5
n_out = 200000

time = np.zeros(n_out)
jp_a,jp_e,jp_l,jp_P,jp_pomega,ast_a,ast_e,ast_l,ast_P,ast_pomega,time
= [np.zeros(n_out) for m in range(11)]
for i,t in enumerate(np.linspace(0, 20000,n_out)):
    sim.integrate(t)
    jp_a[i] = ps[1].a
    jp_P[i] = ps[1].P
    jp_l[i] = ps[1].l
    jp_e[i] = ps[1].e
    ast_pomega[i] = ps[1].pomega

    ast_a[i] = ps[2].a
    ast_P[i] = ps[2].P
    ast_l[i] = ps[2].l
    ast_pomega[i] = ps[2].pomega
    ast_e[i] = ps[2].e
    time[i] = t

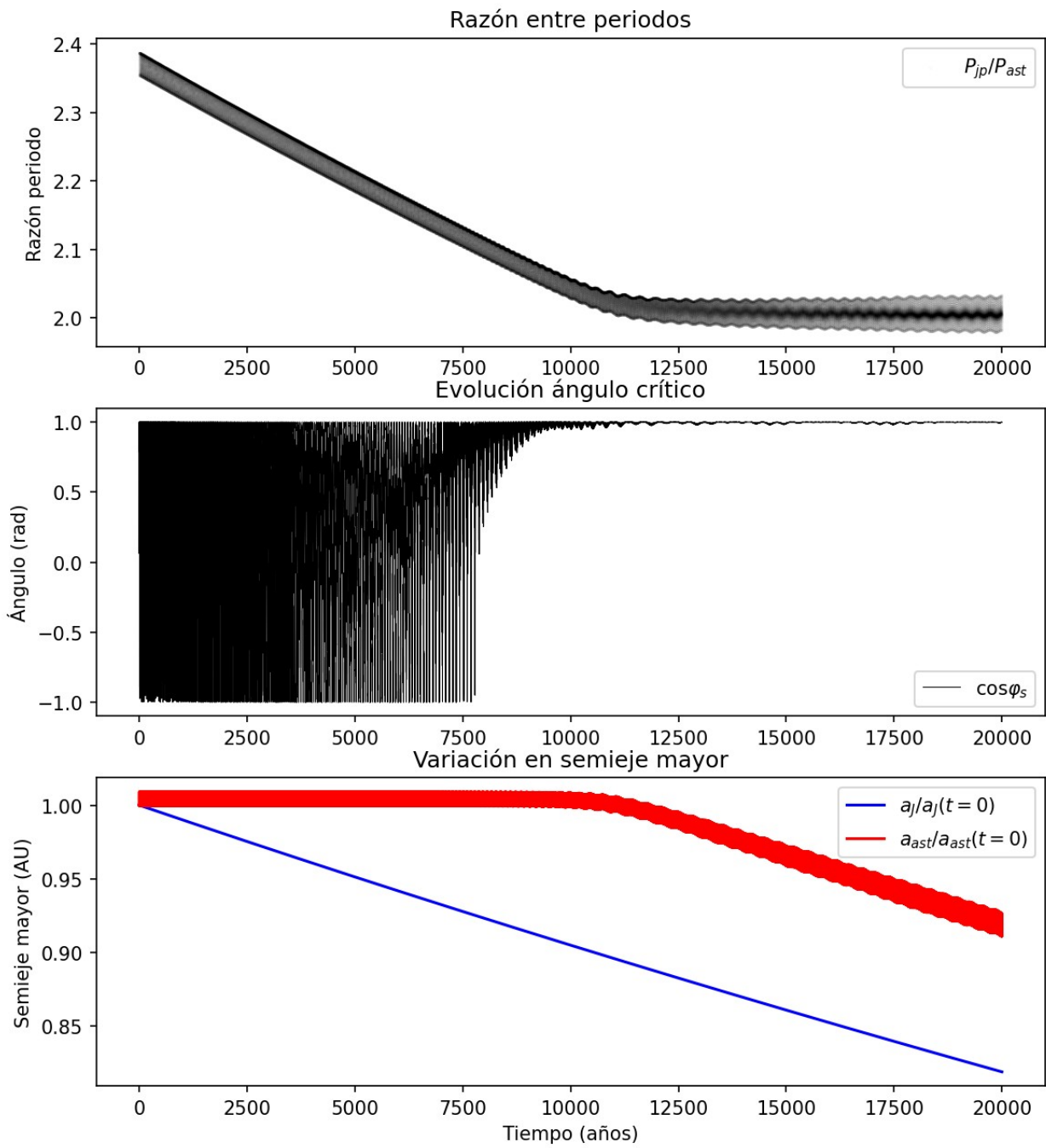
m=2

fig, (ax1,ax2,ax3) = plt.subplots(3,1,figsize=(9,10),dpi=150)
ax1.scatter(time,jp_P/ast_P, label=r'$P_{jp}/P_{ast}$',s=0.0001,c='black')
ax2.plot(time, np.cos((m-1)*ast_l - m*jp_l +
ast_pomega),linewidth=0.5,label=r'$\cos\{\varphi_s\}$',c='black')
```

```

#ax3.set_xlim(0,1000)
ax3.plot(time,jp_a /jp_a[0] ,label=r'$a_{J}/a_{J}(t=0)$',c='blue')
ax3.plot(time,ast_a/ast_a[0],label=r'$a_{ast}/a_{ast}(t=0)$',c='red')
for ax in (ax1,ax2,ax3):
    ax.legend()
ax1.set_ylabel('Razón periodo')
ax2.set_ylabel('Ángulo (rad)')
ax3.set_ylabel('Semieje mayor (AU)')
ax1.set_title('Razón entre periodos')
ax2.set_title('Evolución ángulo crítico')
ax3.set_xlabel('Tiempo (años)')
ax3.set_title('Variación en semieje mayor')
Text(0.5, 1.0, 'Variación en semieje mayor')

```



Podemos ver que luego de los 10000 años se observa resonancia. Luego usando la tercera ley de Kepler encontramos que esto ocurre aproximadamente a los 11700 años. El grado de resonancia corresponde a $m=2$. Una vez capturado el asteroide, el ángulo crítico deja de oscilar. Como es de esperar el semieje mayor del asteroide deja de ser constante una vez capturado y el del Jupiter se mantiene aproximadamente constante.

(b) [3 points] Show the evolution of the eccentricity e_{sat} , the semi-major axis a_{sat} and the pericenter distance $a_{\text{sat}}(1-e_{\text{sat}})$. How far inside needs Jupiter to migrate so the asteroid collides with the Earth?

Aquí para visualizar la colisión hay que ampliar aún mas el rango de tiempo

```
sim = rebound.Simulation()
sim.units = ('yr', 'AU', 'Msun')
sim.integrator = "whfast"
sim.dt = 0.001
sim.add(m=1.)
sim.add(a=5, m=1e-3, e=0.01)
sim.add(a=2.8, m=1e-9)
sim.move_to_com()
ps = sim.particles
rebx = reboundx.Extras(sim)
mod = rebx.load_operator("modify_orbits_direct")
rebx.add_operator(mod)
ps[1].params["tau_a"] = -10**5
n_out = 200000

time = np.zeros(n_out)
jp_a, jp_e, jp_l, jp_P, jp_pomega, ast_a, ast_e, ast_l, ast_P, ast_pomega, time
= [np.zeros(n_out) for m in range(11)]
for i, t in enumerate(np.linspace(0, 40000, n_out)):
    sim.integrate(t)
    jp_a[i] = ps[1].a
    jp_P[i] = ps[1].P
    jp_l[i] = ps[1].l
    jp_e[i] = ps[1].e
    ast_pomega[i] = ps[1].pomega

    ast_a[i] = ps[2].a
    ast_P[i] = ps[2].P
```

```

ast_l[i] = ps[2].l
ast_pomega[i] = ps[2].pomega
ast_e[i] = ps[2].e
time[i] = t

pericenter = ast_a*(1-ast_e)
collision_index = next(x[0] for x in enumerate(pericenter) if x[1] <
1)

fig, ax4 = plt.subplots(1,1,figsize=(9,3),dpi=150)
ax4.plot(time, ast_e, label=r'$e_{ast}$')
ax4.plot(time, ast_a, label=r'$a_{ast}$')

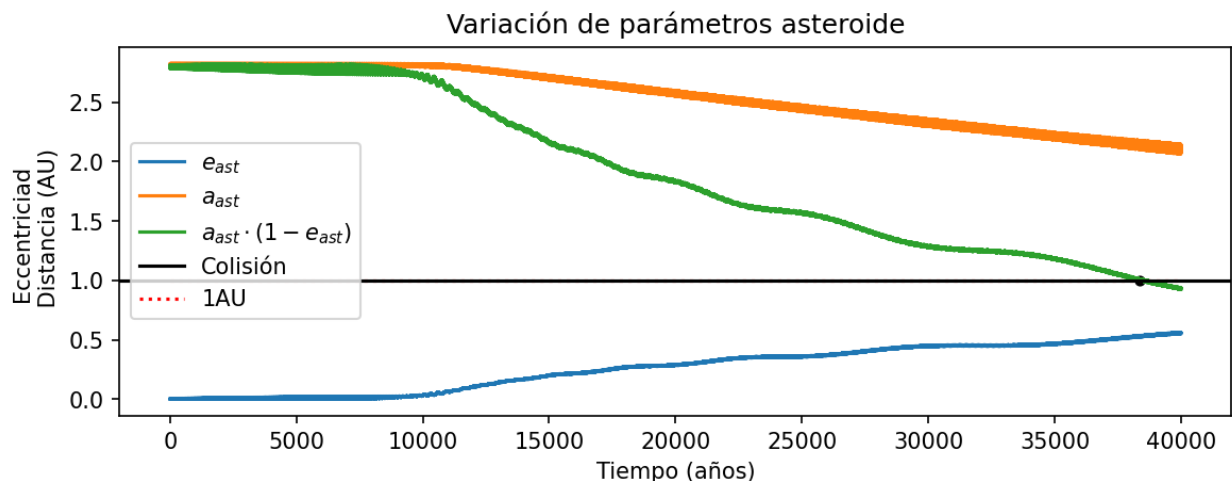
ax4.plot(time, pericenter, label=r'$a_{ast}\cdot(1-e_{ast})$')
ax4.hlines(1,0,time[-1], 'r', label='1AU',linestyles='dotted')
ax4.scatter(time[collision_index],pericenter[collision_index],color='black',s=15)

#ax4.scatter(time[collision_index],pericenter[collision_index],color='
black',s=15)
ax4.axhline(1,c='black',label='Colisión')

ax4.set_xlabel('Tiempo (años)')
ax4.set_ylabel('Eccentricidad \n Distancia (AU)')
ax4.set_title('Variación de parámetros asteroide')
ax4.legend()

<matplotlib.legend.Legend at 0x173dffaf0>

```



```

print(f'Jupiter semimajor axis at collision:
{round(jp_a[collision_index],3)} AU')

```

```
print(f'Asteroid semimajor axis at collision:
{round(ast_a[collision_index],3)} AU')
```

```
Jupiter semimajor axis at collision: 3.408 AU
Asteroid semimajor axis at collision: 2.133 AU
```

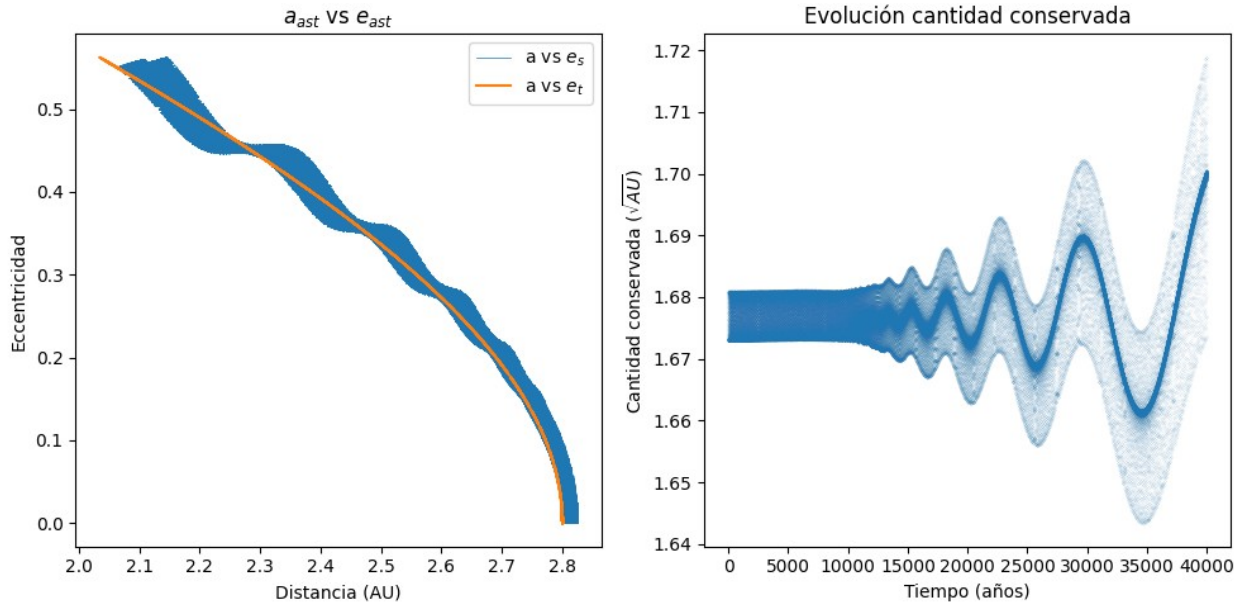
La órbita del asteroide se vuelve cada vez más excéntrica, como es de esperar. Simultáneamente, su semieje mayor disminuye. Ambos cambios son tales que después de la resonancia, el periastro se reduce, alcanzando una distancia de 1 UA antes de los 40.000 años. En este momento, el planeta se encuentra a una distancia de 3.4 UA. A partir de este punto, existe la posibilidad de que ocurran colisiones con la Tierra.

(c) [3 points] Plot of a_{sat} vs e_{sat} , and test whether it follows the conserved quantity seen in class:

$$\sqrt{a} \left(1 + (m-1) \left(1 - (1 - e^2)^{1/2} \right) \right)$$

Use this quantity to calculate analytically the value of a_{J} at which the asteroid would collide with the Earth (i.e., numerical result in b).

```
m=2
const = np.sqrt(2.8)*(1+(m-1)*(1-np.sqrt(1-0**2)))
fig, (ax1,ax2) = plt.subplots(1,2,figsize=(10,5))
ax1.plot(ast_a,ast_e,linewidth=0.5, label = 'a vs $e_{s}$')
ax1.plot(const**2/(1+(m-1)*(1-np.sqrt(1-ast_e**2)))*2,ast_e,label='a
vs $e_{t}$')
ax2.scatter(time, np.sqrt(ast_a)*(1+(m-1)*(1-np.sqrt(1-
ast_e**2))),s=0.001)
ax1.set_xlabel('Distancia (AU)')
ax1.set_ylabel('Eccentricidad')
ax1.set_title(r'$a_{ast}$ vs $e_{ast}$')
ax2.set_xlabel('Tiempo (años)')
ax2.set_ylabel(r'Cantidad conservada $(\sqrt{\text{AU}})$')
ax2.set_title('Evolución cantidad conservada')
ax1.legend()
fig.tight_layout()
```



Se puede ver que la cantidad conservada se mantiene constante previo a la captura del asteroide. Luego el valor comienza a oscilar aumentando su amplitud. En la gráfica de a_{ast} vs e_{ast} se muestra el valor esperado y obtenido para la relación entre ambos valores.

```
def find_min(a,a_0=2.8,e_0=0):
    e = 1-1/a
    const_0 = np.sqrt(a_0)*(1+(m-1)*(1-np.sqrt(1-e_0**2)))
    return (np.sqrt(a)*(1+(m-1)*(1-np.sqrt(1-e**2))) - const_0)**2

ls_a = np.linspace(1,10,100000)
a = np.nanargmin(find_min(ls_a))
print(round(2**(2/3)*ls_a[a],3))
print(round(ls_a[a],3))

3.359
2.116
```

El eje semimayor de Jupiter en la colision es de 3.359 AU (muy cercano al valor calculado en [b])

El eje semimayor del asteroide en la colision es de 2.116 AU

(d) Outwards migration [Bonus of up to 4 points]. Now set a Jupiter that migrates outwards as:

$$a_J(t) = 4 \text{ au} \exp(+t/\tau_a)$$

with $\tau_a = 10^5$ yr. Keep all other parameters the same.

Comment on the behavior of the resonance encounter ($\Delta=0$): changes on a , e and the conserved quantity above. Is there a capture?